UseCase : Potential Initial Access via DLL Search Order Hijacking

Description:

This detection is designed to identify potential Initial Access activity via DLL Search Order Hijacking (MITRE ATT&CK T1574.001). Adversaries may place malicious DLLs in non-standard directories that are loaded by legitimate system processes, allowing them to execute code with elevated privileges or maintain persistence. By monitoring for DLLs outside standard Windows paths that are loaded by system processes, this logic flags suspicious file activity that could indicate early-stage compromise or malicious lateral movement attempt..

SPL Search:

```
| tstats
   summariesonly=true
   min(_time) AS firstTime
   max(_time) AS lastTime
 count(Filesystem.file_name) AS file_count
 FROM datamodel=Endpoint.Filesystem
 BY Filesystem.dest
    Filesystem.file_name
    Filesystem.user
    Filesystem.file_path
    Filesystem.process_guid
| rename Filesystem.* AS *
| join process_guid
   [
      | tstats
         summariesonly=true
         values(Processes.process_name) AS process_name
         values(Processes.process_path) AS process_path
         values(Processes.parent_process_guid) AS parent_process_guid
        FROM datamodel=Endpoint.Processes
        BY Processes.process_guid
      | rename Processes.* AS *
   ]
```
```
| join process_guid
          [
             search index=winevents (EventCode=1 OR EventCode=4688)
             | eval process_command_line=coalesce(CommandLine, Process_Command_Line)
             | table process_guid process_command_line
          ]
```
```
| join parent_process_guid
   [
      | tstats
         summariesonly=true
         values(Processes.process_name) AS parent_process_name
         values(Processes.process_path) AS parent_process_path
```

```
        FROM datamodel=Endpoint.Processes
        BY Processes.process_guid
      | rename Processes.* AS *
      | rename process_guid AS parent_process_guid
  ]
```
```
| join parent_process_guid
  [
        search index=winevents (EventCode=1 OR EventCode=4688)
        | table parent_process_guid CommandLine
  ]
```
```
| where like(file_name,"%.dll")
| where NOT (
  like(file_path,"C:\\Windows\\System32%")
  OR like(file_path,"C:\\Windows\\SysWOW64%")
  OR like(file_path,"C:\\Program Files%")
  OR like(file_path,"C:\\Program Files (x86)%")
)
| where match(process_path, "(?i)\\\\windows\\\\system32\\\\")
| eval Time=if(
  firstTime==lastTime,
  strftime(firstTime,"%Y-%m-%d %H:%M:%S"),
  strftime(firstTime,"%Y-%m-%d %H:%M:%S")." - ".strftime(lastTime,"%Y-%m-%d %H:%M:%S")
)
| eval FileModified=if(firstTime!=lastTime,"Yes","No")
| rename dest as Host ,
      file_name as DLL_Name,
      user as User,
      file_path as DLL_Path,
      parent_process_name as ParentProcess,
      parent_process_path as ParentProcess_Path,
      process_name as ProcessName,
      process_path as Process_Path,
      ```process_command_line as Process_CommandLine```
| table Time Host DLL_Name User FileModified DLL_Path ProcessName Process_Path ParentProcess
ParentProcess_Path
| sort -Time
```

Key Detection Pillars:

- **Trust Boundary Violation:** Trusted parent process (System32) loading untrusted DLL
- **Search Order Abuse:** Exploitation of Windows DLL loading precedence
- **Contextual Correlation:** Enriched with process lineage, command line, and temporal analysis
- **Low Noise Design :** Strong focus on System32 processes to significantly reduce false positives
```

False Positive Considerations :

- **Rare legitimate cases :** Legacy applications with custom DLLs or specific development tools.
- **Tuning recommendations:** Create an allowlist for approved `DLL_Path` + `ParentProcess` combinations specific to your environment.

Analyst Guidance (Investigative Steps):

1. **Immediate Triage**
    - Validate DLL_Path — focus on %Temp%, %AppData%, %ProgramData%, Downloads, and user profile directories.
    - Review FileModified field. Recent modification strongly suggests a dropper.
2. **Process Lineage Analysis**
    - Examine full process tree using process_guid.
    - Identify how the parent process (System32 binary) was spawned — look for unusual parent-child relationships.
3. **Artifact Collection**
    - Hash the suspicious DLL and search on VirusTotal, Hybrid-Analysis, and internal threat intel.
    - Collect command line arguments and check for obfuscation patterns or Base64.
4. **Pivot & Scope**
    - Correlate with network connections from the same process_guid.
    - Check for subsequent persistence mechanisms (Registry Run keys, scheduled tasks, WMI).
5. **Forensic Recommendations**
    - Memory dump of the suspicious process (using Procdump or similar).
    - Full disk forensics if initial findings confirm malicious activity.

Detection Maturity :

This rule is at **Medium-High** maturity level. Can be further enhanced with:

- Integration with Sysmon Event ID 7 (Image Loaded)
- Baseline of normal DLL loading behavior per host

Mohamad Mahdi Nabizadeh